

COMP30026

Models of Computation

Lecture 19: TM Variants and Decidable Languages

Cezary Kaliszyk and William Umboh

Semester 2, 2026

Announcements

Please fill in Small-Class Teacher Survey
A2 out today on Ed

Where are we?

Last time:

Turing machines

- Recognizers and deciders
- Turing-recognizable and Turing-decidable languages
- Church-Turing Thesis

Today:

Equivalence of variants of the Turing machine model

- Multi-tape TMs
- Nondeterministic TMs

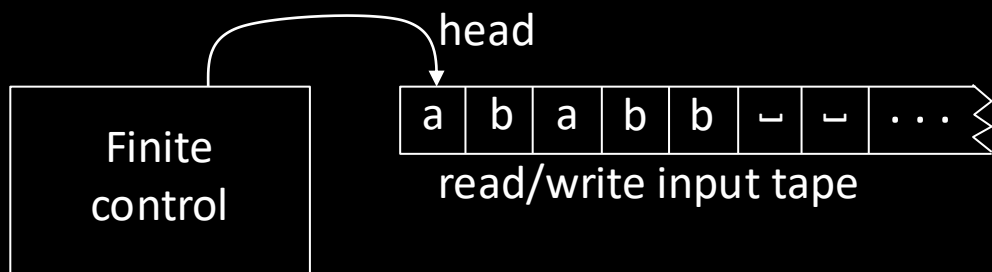
Notation for encodings and TMs

Decision procedures for automata and grammars

Technique: Simulation



Turing machine model – review



On input w a TM M may halt (enter q_{acc} or q_{rej}) or loop (run forever).

So M has 3 possible outcomes for each input w :

1. Accept w (enter q_{acc})
2. Reject w by halting (enter q_{rej})
3. Loop (running forever)

Satisfiability of Prop Logic: Decidable

Satisfiability of First-Order Logic: Undecidable (but unsatisfiability is recognizable)

A is T-recognizable if $A = L(M)$ for some TM M .

A is T-decidable if $A = L(M)$ for some TM decider M .
halts on all inputs ↗

Turing machines model general-purpose computation.

Q: Why pick this model?

A: C-T Thesis: Choice of model doesn't matter.

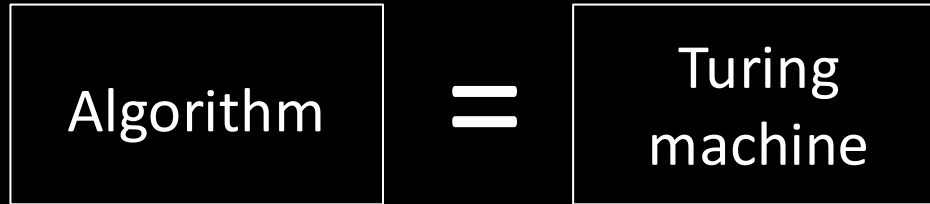
All reasonable models are equivalent in power.

Virtues of TMs: simplicity, familiarity (similar to calculation using pencil and paper).

Church-Turing Thesis ~1936 - review



Alonzo Church
1903–1995

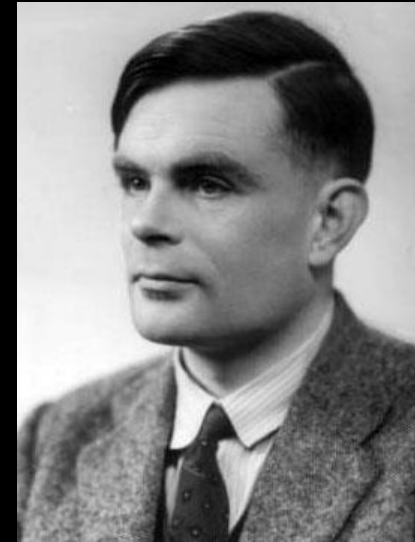


Intuitive

Formal

Turing machine can solve any computational problem that can be solved by any “reasonable” model of unrestricted computation:
 λ -calculus, random access machine, your favorite programming language, ...

To show a problem is uncomputable, suffices to show that no TM can decide it

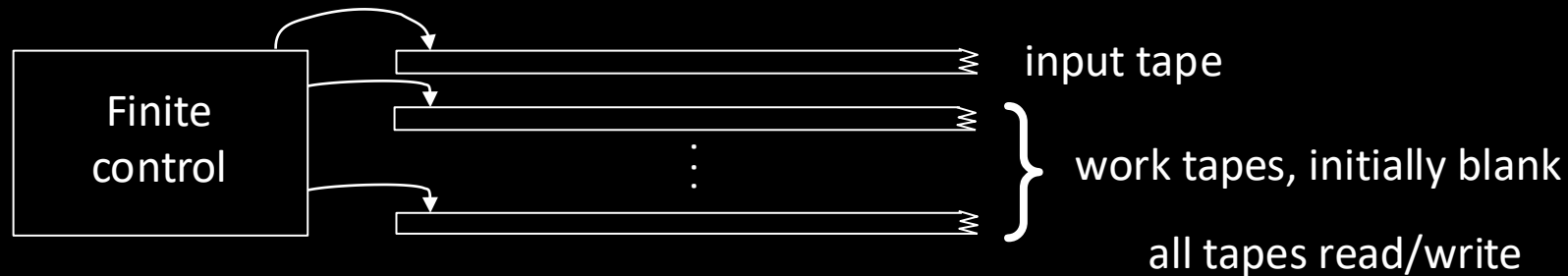


Alan Turing
1912–1954

Next: Some evidence of Church-Turing Thesis

- “Reasonable” extensions of TMs do not increase computational power
- Regular TM can simulate them

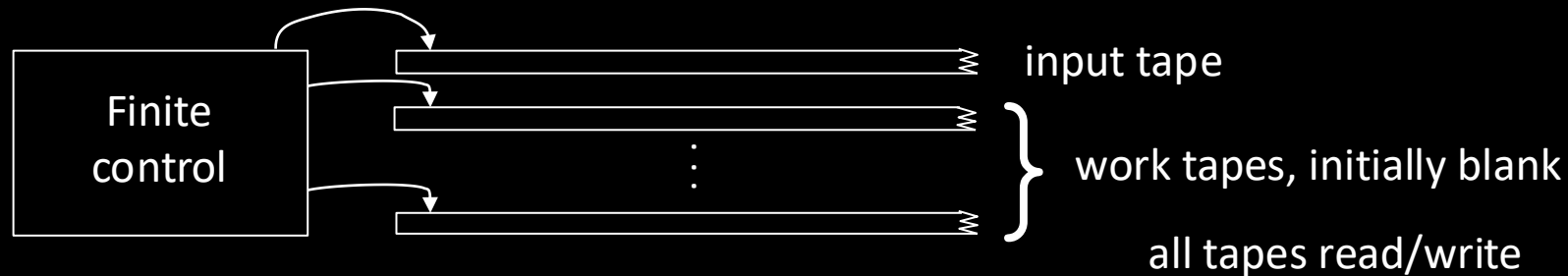
Multi-tape Turing machines



What's new:

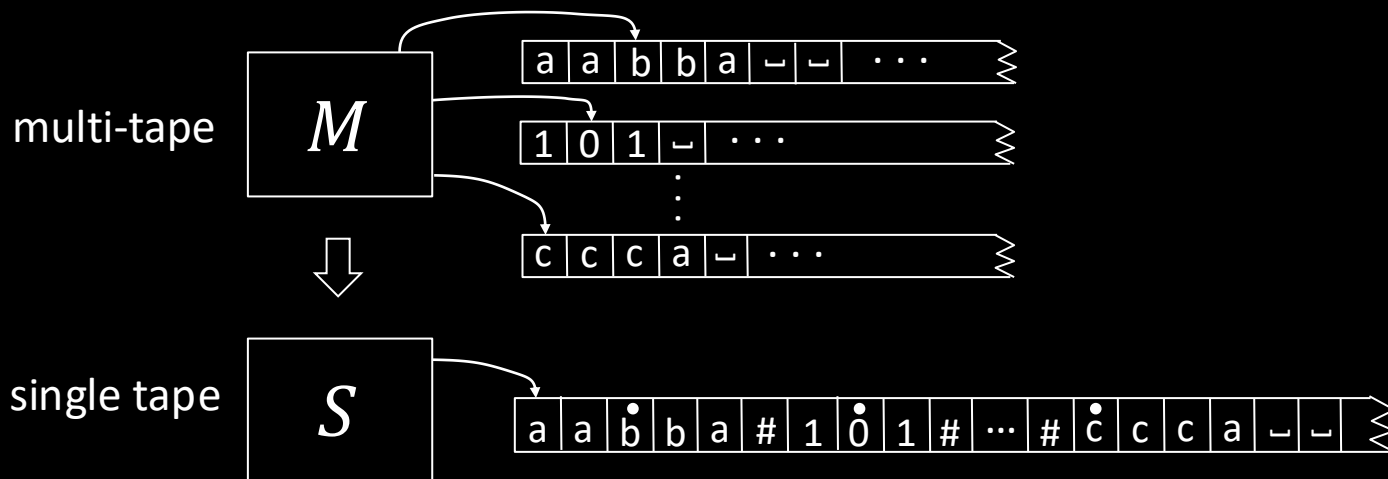
- Fixed number of work tapes (no new work tape can be created)
- Initially work tapes are blank (all $_$) and tape heads at start
- Transition function depends on all tape heads and moves all tape heads in a single step

Multi-tape Turing machines



Theorem: A is T-recognizable iff some multi-tape TM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert multi-tape to single tape:



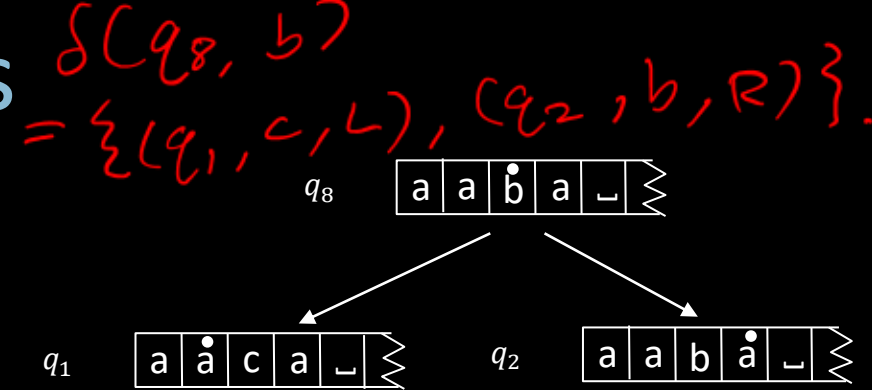
S simulates M by storing the contents of multiple tapes on a single tape in "blocks". Record head positions with dotted symbols.

Some details of S :

- 1) To simulate each of M 's steps
 - a. Scan entire tape to find dotted symbols.
 - b. Scan again to update according to M 's δ .
 - c. Shift to add room as needed.
- 2) Accept/reject if M does.

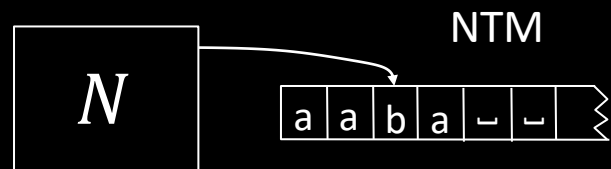
Nondeterministic Turing machines

A Nondeterministic TM (NTM) is similar to a Deterministic TM except for its transition function $\delta: Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$ and accepts if and only if some branch accepts

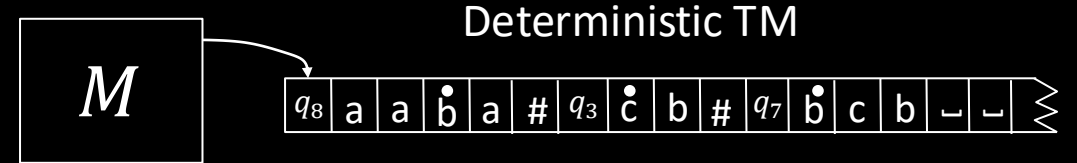
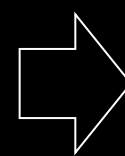
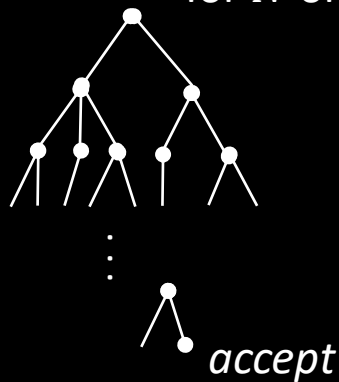


Theorem: A is T-recognizable iff some NTM recognizes A

Proof: $(\rightarrow)$ immediate. $(\leftarrow)$ convert NTM to Deterministic TM.



Nondeterministic computation tree for N on input w .



M simulates N 's computation tree using BFS:

- store each branch's tape in a separate "block" on its tape.
- store the head location, and the state for each branch, in the block.

To simulate each step of N :

- Apply δ to each block, left to right
- If a branch forks, then M copies the block and shifts to make room
- If a branch accepts then M accepts.

TM Variants: Summary

- Multi-Tape TM = Single-Tape TM
- Nondeterministic TM = Deterministic TM
- Additional features are “syntactic sugar”: adds convenience not computational power
- To show language is decidable/recognizable, suffices to give multi-tape or nondeterministic TM

Decision Problems for Computational Models

- Given a program's source code, does it correctly implement a specification?
- **Acceptance Problem:** Given a FA/grammar/TM and string w , does it accept/generate w ?
 - Application: Correctness on test cases
- **Equivalence Problem:** Given two FA/grammars/TMs, are their languages equal?
 - Application: Equivalence with known correct solution
 - Application: Is refactored code equivalent to previous code?
- **Emptiness Problem:** Given a FA/grammar/TM, is its language empty?
 - Special case of equivalence problem
 - Application: Is there an input that triggers an exception?

Notation for encodings and TMs

Notation for encoding objects into strings

- If O is some object (e.g., polynomial, automaton, graph, etc.), we write $\langle O \rangle$ to be an encoding of that object into a string. Every discrete object has a string representation!
- If $O_1, O_2, \dots, O_k$ is a list of objects then we write $\langle O_1, O_2, \dots, O_k \rangle$ to be an encoding of them together into a single string.

Notation for writing Turing machines

We will use high-level English descriptions of algorithms when we describe TMs, knowing that we could (in principle) convert those descriptions into states, transition function, etc. Our notation for writing a TM M is

$M =$ “On input w
[English description of the algorithm]”

Acceptance Problem for DFAs

Let $A_{\text{DFA}} = \{\langle B, w \rangle \mid B \text{ is a DFA and } B \text{ accepts } w\}$

Theorem: A_{DFA} is decidable

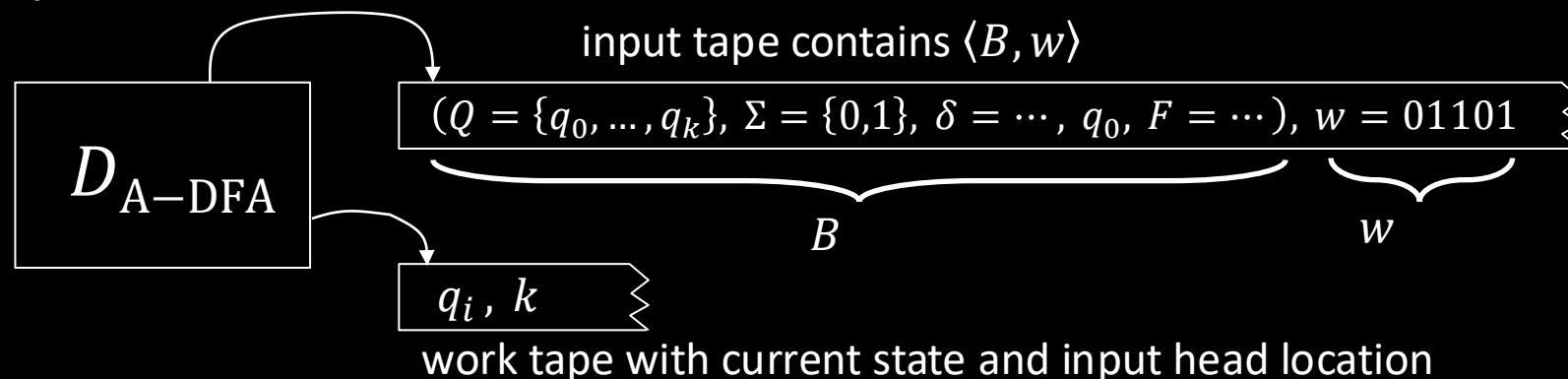
Proof: Give TM $D_{\text{A-DFA}}$ that decides A_{DFA} .

$D_{\text{A-DFA}}$ = "On input s

1. Check that s has the form $\langle B, w \rangle$ where B is a DFA and w is a string; *reject* if not.
2. Simulate the computation of B on w .
3. If B ends in an accept state then *accept*.
If not then *reject*."

Shorthand:

On input $\langle B, w \rangle$



Acceptance Problem for NFAs

Let $A_{\text{NFA}} = \{\langle B, w \rangle \mid B \text{ is a NFA and } B \text{ accepts } w\}$

Theorem: A_{NFA} is decidable

Proof: Give TM $D_{\text{A-NFA}}$ that decides A_{NFA} .

$D_{\text{A-NFA}} =$ “On input $\langle B, w \rangle$

1. Convert NFA B to equivalent DFA B' .
2. Run TM $D_{\text{A-DFA}}$ on input $\langle B', w \rangle$. [Recall that $D_{\text{A-DFA}}$ decides A_{DFA}]
3. *Accept* if $D_{\text{A-DFA}}$ accepts.
Reject if not.”

New technique: Reduction

Use conversion construction and previously constructed TM as a subroutine.

We say that we reduce A_{NFA} to A_{DFA}

Emptiness Problem for DFAs

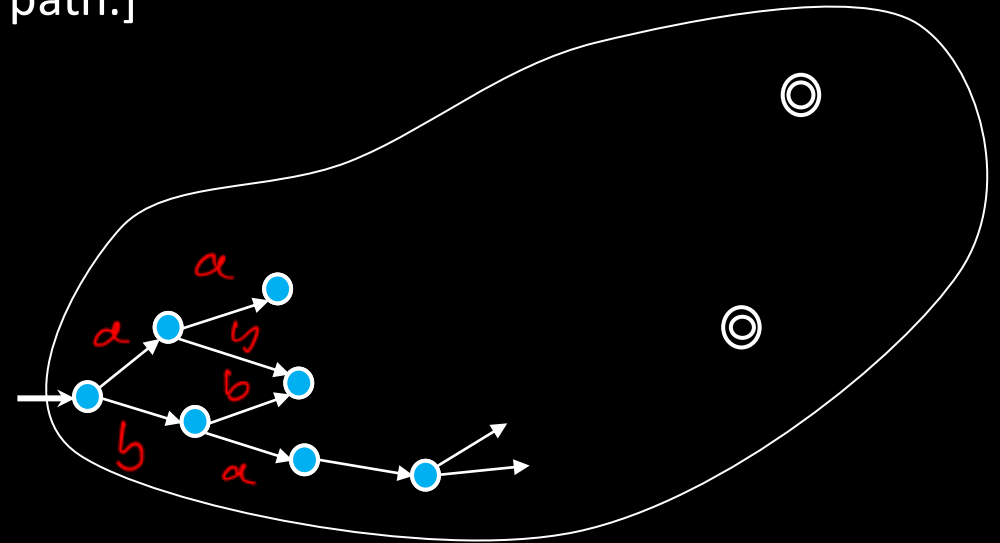
Let $E_{\text{DFA}} = \{\langle B \rangle \mid B \text{ is a DFA and } L(B) = \emptyset\}$

Theorem: E_{DFA} is decidable

Proof: Give TM $D_{E-\text{DFA}}$ that decides E_{DFA} .

$D_{E-\text{DFA}}$ = "On input $\langle B \rangle$ [IDEA: Input string = computation path.]

1. **Mark** start state.
2. Repeat until no new state is marked:
Mark every state that has an incoming arrow from a previously marked state.
3. *Accept* if no accept state is marked.
Reject if some accept state is marked."



BFS of state transition diagram

Equivalence problem for DFAs

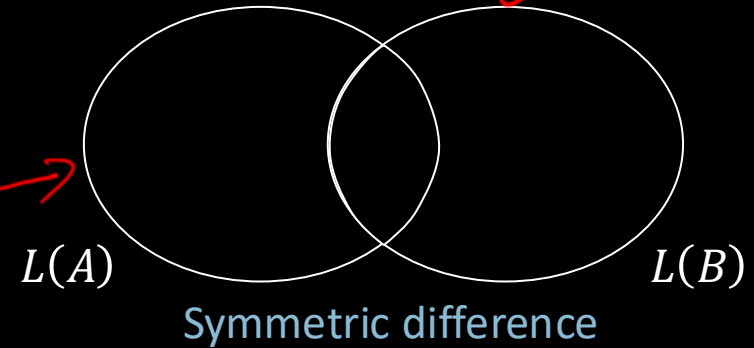
Let $EQ_{DFA} = \{\langle A, B \rangle \mid A \text{ and } B \text{ are DFAs and } L(A) = L(B)\}$

Theorem: EQ_{DFA} is decidable

Proof: Give TM D_{EQ-DFA} that decides EQ_{DFA} .

$D_{EQ-DFA} =$ "On input $\langle A, B \rangle$ [IDEA: Make DFA C that accepts w where A and B disagree and check for emptiness]

1. Construct DFA C where $L(C) = \left(L(A) \cap \overline{L(B)} \right) \cup \left(\overline{L(A)} \cap L(B) \right)$.
2. Run D_{E-DFA} on $\langle C \rangle$.
3. *Accept* if D_{E-DFA} accepts.
Reject if D_{E-DFA} rejects."



Note: $\bar{L} = L^c$, i.e. the complement of L

Decision Problems for Computational Models

	Finite Automata	Context-Free Grammars	Turing Machines
Acceptance	Decidable	Decidable	???
Emptiness	Decidable	Decidable	???
Equivalence	Decidable	Undecidable	???

Let $AMBIG_{CFG} = \{ \langle G \rangle \mid G \text{ is an ambiguous CFG} \}$

Theorem: $AMBIG_{CFG}$ is NOT decidable

Upshot for checking correctness of FAs and CFGs:

- Checking correctness of FA is easy (no false positives)
- Can only check correctness of CFG on test cases (possible false positives)

Quick review of today

Equivalence of variants of the Turing machine model

- Multi-tape TMs
- Nondeterministic TMs

Notation for encodings and TMs

Decision procedures for automata and grammars

Key ideas:

- Simulation
- Reduction
 - NFA Acceptance to DFA Acceptance
 - DFA Equivalence to DFA Emptiness